Individual reflection: median-of-medians

Ethan Fernandez, Programming Assignment 2, Advanced Algorithms

Worst case

With three-way partitioning, median-of-medians has no input that makes it worse than linear. The pivot is guaranteed to have about 3n/10 keys on each side, so the recurrence $T(n) \le T(n/5) + T(7n/10) + O(n)$ gives at most 40n comparisons, and the worst inputs can only change the constant. The most expensive input for the group sorts is reverse-sorted data, because insertion sort on a reversed group of five costs 10 comparisons where a sorted group costs 4 and a random group about 7.7. I did not benchmark reverse-sorted input, but the measured costs sit far below the bound, at about 10.3 comparisons per element on random input and 8.96 on sorted input. The input that does break it is all-equal keys under Lomuto partitioning, which was also my hardest bug.

Hardest bug

My hardest bug was the Lomuto blow-up. The first version, adapted with Claude, used Lomuto partitioning and Claude reported it as passing every test, but when I ran the tests myself in a Windows Debug build they had still not finished after more than 15 minutes, and I stopped them because something was obviously wrong. The cause was the all-equal test input. Lomuto with ≤ sends every key equal to the pivot to the left, so each round removes only one element, and worse, the recursive call that finds the median of the n/5 medians runs on all-equal medians and hits the same problem at every level of the recursion. My duplicates experiment later measured comparison slopes rising from 3.37 to 3.56 to 4.13, so the cost grows faster than any polynomial. Switching to three-way partitioning removed every key equal to the pivot, and the same tests then finished almost instantly.

What I would do differently

I would question the assumptions behind the 3n/10 argument before implementing it. I treated it as unconditional when it relies on keys equal to the pivot being split off, which a careful reading would have shown me before the tests did. I would also start with three-way partitioning rather than adding it as a fix. Finally, I would default to groups of seven instead of five, since g = 7 was slightly faster at $n = 10^7$ (40.4 against 42.6 ns per element) because it gathers and recurses on fewer medians.

What I now understand

The main thing I understand now is what a worst-case guarantee costs in practice. On random input median-of-medians was 2.89 times slower than quickselect at $n = 10^7$, even though it made only about twice as many comparisons, because every round pays for the group sorts, gathering the medians and the recursive call whether or not they were needed. Quickselect's bad cases never came close in 2,000 seeds, so the guarantee is insurance paid on every run against something that almost never happens, which is why nth_element only falls back to a slower guaranteed method when its recursion gets too deep. I also now see why the guarantee needs ties split, since the 3n/10 argument counts keys equal to the pivot on both sides and Lomuto puts them all on one, and why g

= 3 fails, since 1/3 + 2/3 = 1 leaves the recurrence no room to shrink and my comparisons per element kept rising from 10.3 to 14.1 as n grew.